~ Complete Step by Step ~

Express.js

Routing • Middleware • REST APIs • Diagrams

Index

1 Introduction to Express
16 File Upload (Multer)
2 Setup & First App
17 Database Integration
3 How Express Works
18 Authentication & Authorization
4 Routing
19 Security
5 Params, Query & Body
20 Project Structure & MVC
6 Request & Response Objects
21 App Settings & Advanced Topics
7 Express Router
22 Logging & Debugging
8 Middleware
23 Testing
9 Built-in Middleware & Static Files
24 Performance & Production
10 Third-party Middleware
25 Deployment
11 Error Handling
26 Express 5 & TypeScript
12 Template Engines
27 Mini Project: Bookstore API
13 Forms, Cookies & Sessions
28 Interview Q&A
14 REST API & CRUD
29 Cheat Sheet
15 Validation

How to use

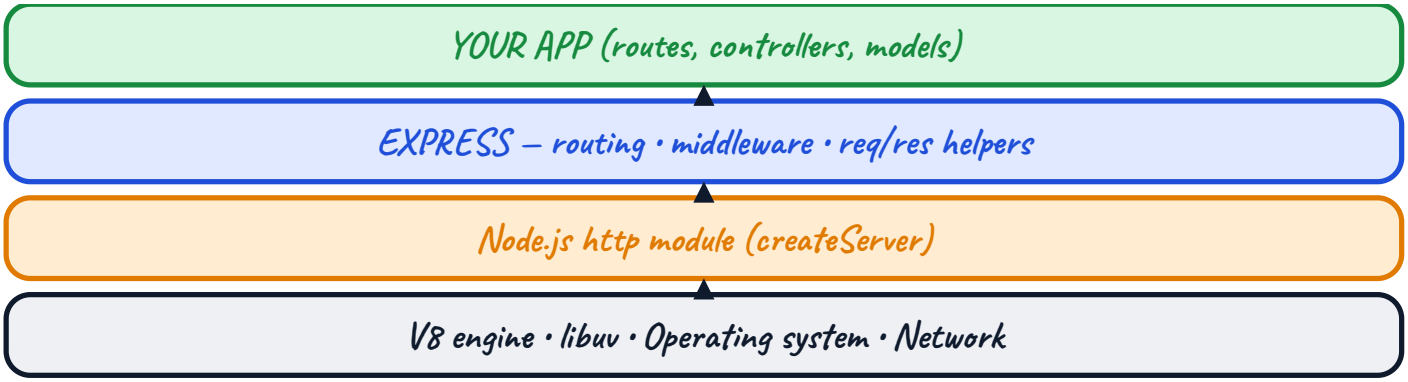
- Needs **JavaScript** and basic **Node.js** (modules, npm, async/await). Examples use **ES modules** (***import***) with **Express 4/5**.
- Type each example, run it with ***node --watch app.js***, and test with **Postman / Thunder Client / curl**. Redraw the diagrams in your notebook.

Notes by asadokz18 ✿

1. Introduction to Express.js

Definition

Express.js is a **fast, minimal and flexible web application framework for Node.js**. It adds **routing, middleware** and helpful **request/response methods** on top of Node's built-in ***http*** module so you can build **websites and REST APIs** with little code. Created by **TJ Holowaychuk (2010)**, now maintained by the OpenJS Foundation. It is the "E" in the **MERN / MEAN** stacks.



Why use Express? (features)

- → **Routing** – map URL + method to a function
- → **Middleware** – plug-in pipeline (auth, logs, parsing)
- → **Simple & unopinionated** – you choose structure & tools
- → **Template engines** – EJS, Pug, Handlebars
- → **Static files** – one line to serve a folder
- → **Huge ecosystem** – thousands of middleware
- → **JSON-friendly** – ***res.json()***
- → **Fast** to learn; used by Uber, IBM, Accenture, X

Raw Node http vs Express – same job

Node http module

```
http.createServer((req, res) => {
  const url = new URL(req.url, "http://x");
  if (req.method === "GET" && url.pathname === "/users") {
    res.writeHead(200, {"Content-Type":"application/json"});
    res.end(JSON.stringify(users));
  } else {
    res.writeHead(404); res.end("Not found");
  }
}).listen(3000);   // + manual body parsing, etc.
```

Express

```
const app = express();

app.get("/users", (req, res) => {
  res.json(users);          // sets header + stringify
});

// 404, JSON body, params, static files...
// all built in or one line away
app.listen(3000);
```

Framework	Notes
Express	Minimal, most popular, huge community, you assemble the pieces
Fastify	Faster, schema validation built in, plugin system
Koa	By Express creators, modern async/await, tiny core
NestJS	Full opinionated framework (TypeScript, Angular-style) – runs on Express/Fastify

Good for

REST APIs, web apps (SSR), proxies, microservices, prototypes, backends for React / Vue / mobile apps.

Versions

Express 4 (long the standard) and **Express 5** (stable, default on npm now: async errors handled automatically, stricter path syntax). See Chapter 26.

2. Setup & Your First Express App

Install

```
node -v && npm -v                       # Node 20+ recommended
mkdir my-app && cd my-app
npm init -y                             # creates package.json
npm pkg set type=module                 # use import/export (ES modules)
npm install express                     # production dependency
npm install -D nodemon                  # auto-restart in development (or: node --watch)
```

app.js – Hello Express

```
import express from "express";          // CommonJS: const express = require("express");

const app = express();                  // 1. create the app
const PORT = process.env.PORT || 3000;

app.get("/", (req, res) => {            // 2. define a route: METHOD + PATH + HANDLER
  res.send("Hello from Express 👋");
});

app.listen(PORT, () => {                // 3. start the server
  console.log(`Server running at http://localhost:${PORT}`);
});
```

```
node app.js          # or  nodemon app.js   |   node --watch app.js
# open http://localhost:3000      Stop: Ctrl + C
```

package.json – scripts

```
"scripts": { "start": "node app.js", "dev": "nodemon app.js" }          // npm start   |   npm run dev
```

The 3 building blocks in the code

Part	Meaning
`express()`	Factory function → returns the **application** object ***app***
`app.get(path, handler)`	Route: when a **GET** request arrives at **path**, run **handler(req, res)**
`app.listen(port, cb)`	Binds & listens (wraps ***http.createServer(app).listen()***; cb runs when ready

Several routes + JSON + status codes

```
app.get("/about", (req, res) => res.send("<h1>About page</h1>"));                 // HTML string
app.get("/api/hello", (req, res) => res.json({ message: "Hello", time: Date.now() }); // JSON
app.get("/old", (req, res) => res.redirect(301, "/about"));                       // redirect
app.get("/secret", (req, res) => res.status(403).send("Forbidden"));              // status code
app.use((req, res) => res.status(404).send("Page not found"));                    // 404 (must be LAST)
```

Common first-day errors

- ***EADDRINUSE*** – port already used: change PORT or stop the other app.
- ***Cannot GET /x*** – no route for that path/method.
- ***Cannot use import*** – add ***"type":"module"***.
- Browser keeps loading – you forgot ***res.send()*** / ***next()***.

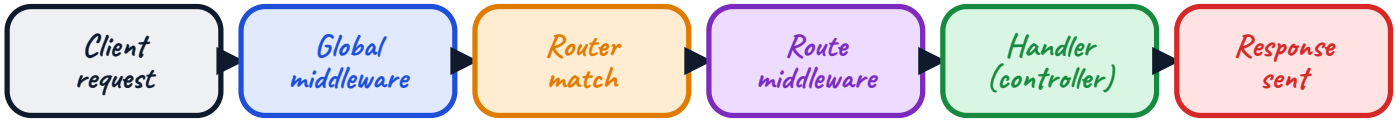
Developer tools

- **Postman / Thunder Client / Insomnia** – send POST, PUT, DELETE
- ***curl -i localhost:3000*** – quick tests
- VS Code **REST Client** (.http files)
- Express generator: ***npx express-generator*** (old scaffold)

3. How Express Works – The Request-Response Cycle

Every HTTP request flows through the same pipeline. Understanding it explains routing, middleware and errors.

REQUEST – RESPONSE CYCLE



if no route matches → 404 handler if something throws / next(err) → error middleware (err, req, res, next)

1. The browser/app sends an **HTTP request** (method + URL + headers + optional body).
2. Node gives Express the raw ***req*** and ***res***; Express **enhances** them with extra methods (***res.json***, ***req.params***…).
3. The request passes through **middleware** in the order they were registered (***app.use***).
4. The **router** finds the first route whose **method and path** match and runs its handler(s).
5. A handler must **end the cycle** by sending a response (***res.send / json / end / render / redirect***) – otherwise the client waits forever.

The app object (the heart of Express)

Method	Purpose
app.get / post / put / patch / delete(path, ...fn)	Define routes for HTTP methods (***app.all*** = every method)
app.use([path], ...middleware)	Mount middleware / routers (any method; prefix match)
app.route(path)	Chain several methods on one path
app.set(name, value) / app.get(name)	Settings: "***view engine***", "***trust proxy***", "***port***"
app.listen(port, cb)	Start HTTP server · ***app.locals*** = global template vars

req (IncomingMessage)
req.params /:id
req.query ?a=1
req.body JSON / form
req.headers / req.get()
req.cookies · req.ip · req.method
req.path · req.originalUrl · req.protocol

res (ServerResponse)
res.send() · res.json() · res.end()
res.status(code)
res.redirect() · res.render()
res.sendFile() · res.download()
res.set() · res.cookie()
res.type() · res.format() · res.location()

Inside a handler – a full tour

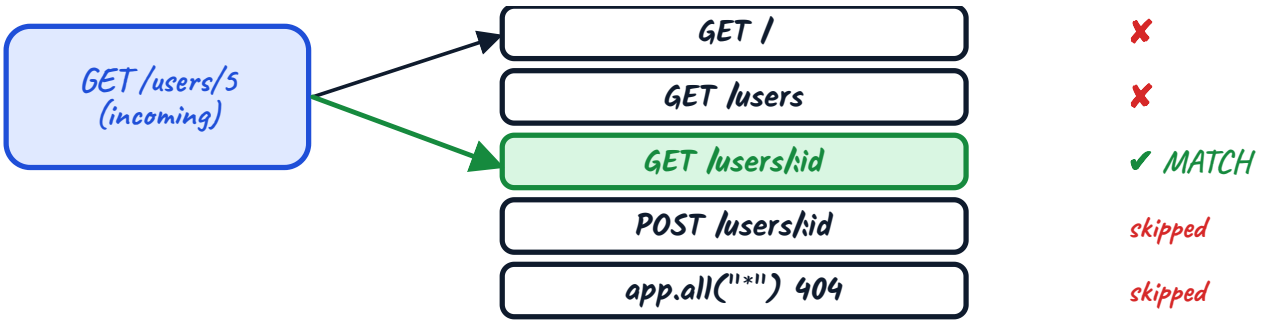
```
app.post("/api/users/:id", (req, res) => {
  console.log(req.method, req.originalUrl);              // POST /api/users/5?x=1
  console.log(req.params, req.query, req.body);          // {id:"5"} {x:"1"} {name:"Asad"}
  console.log(req.get("Content-Type"), req.ip);          // header read, client ip
  res.status(201).set("X-Powered-By-Me", "asad").json({ ok: true });   // chainable response
});
```

Remember

Handlers can be **sync or async**. Sending a response **twice** throws ***ERR_HTTP_HEADERS_SENT*** – use ***return res.json(...)*** to stop the function after replying.

4. Routing

Routing decides how the app answers a request to a particular **endpoint** (URL path + HTTP method). Syntax: ***app.METHOD(PATH, HANDLER)***



Routes are checked TOP → BOTTOM; the first match handles the request (order matters!)

```
app.get("/", (req, res) => res.send("Home"));                          // READ
app.post("/users", (req, res) => res.status(201).send("Created"));     // CREATE
app.put("/users/:id", (req, res) => res.send("Replaced"));             // REPLACE
app.patch("/users/:id", (req, res) => res.send("Updated"));            // PARTIAL UPDATE
app.delete("/users/:id", (req, res) => res.sendStatus(204));           // DELETE (sendStatus sets status + text)
app.all("/secret", (req, res, next) => { console.log("any method"); next(); });   // all methods
app.use("/api", apiRouter);                                            // any method, prefix "/api..."
```

Route paths – strings & patterns

Path	Matches
`"/about"`	exactly /about
`"/users/:id"`	/users/5, /users/abc → **req.params.id**
`"/users/:id/posts/:postId"`	several params
`"/files/*splat"` (Express 5) / `"/files/*"` (v4)	wildcard – rest of the path
`"/ab?cd"` · `"/ab+cd"` · `"/ab*cd"` (v4)	optional / repeated / any chars (Express 5 uses `{"{/...}"}` for optional)
`/\/api\/v\d+\/users/` (RegExp)	any path matching the regular expression
`"/users/:id(\\d+)"` (v4)	param restricted to digits (v5: validate in code / param middleware)
`["/home", "/index"]`	array of paths → same handler

Multiple handlers, route chaining & order

```
const check = (req, res, next) => { console.log("checking"); next(); };          // route-level middleware
app.get("/dashboard", check, (req, res) => res.send("Dashboard"));               // many functions: [a, b, c] also works

app.route("/books")                                                              // chain methods on one path
  .get((req, res) => res.json(books))
  .post((req, res) => res.status(201).json(req.body));

app.get("/users/new", (req, res) => res.send("form"));        // specific route FIRST ...
app.get("/users/:id", (req, res) => res.send("user"));        // ... before the dynamic one, or "new" is taken as an :id!
app.use((req, res) => res.status(404).json({ error: `Cannot ${req.method} ${req.originalUrl}` }));   // catch-all LAST
```

next("route") skips the remaining handlers of the current route. Routing is **case-insensitive** and ignores a trailing slash by default (***app.set("case sensitive routing", true)*** changes it).

5. Route Params, Query Strings & Request Body

Clients send data to the server in **four places**. Choose the right one:



Where	Express	Use for
URL path /users/5	**req.params**	**Identify** a resource (id, slug)
Query string ? q=js&page=2	**req.query**	Filter, search, sort, pagination (optional data)
Body (POST/PUT/PATCH)	**req.body**	Create / update data: JSON, form, files (needs parser middleware)
Headers & cookies	**req.headers / req.get() / req.cookies**	Token, content type, language, session

1. Route parameters req.params

```
app.get("/users/:userId/books/:bookId", (req, res) => {
  const { userId, bookId } = req.params;                    // always STRINGS → convert: Number(userId)
  res.json({ userId, bookId });                             // GET /users/34/books/8989
});
app.param("id", (req, res, next, id) => {                   // runs for every route containing :id
  if (!/^\d+$/.test(id)) return res.status(400).json({ error: "id must be a number" });
  req.id = Number(id);  next();
});
```

2. Query string req.query

```
app.get("/api/products", (req, res) => {                    // GET /api/products?category=phone&min=100&sort=-price&page=2&limit=5
  const { category, min = 0, sort = "name", page = 1, limit = 10 } = req.query;      // values are strings (or arrays: ?tag=a&tag=b)
  let list = products.filter(p => (!category || p.category === category) && p.price >= Number(min));
  const start = (page - 1) * limit;
  res.json({ page: +page, total: list.length, data: list.slice(start, start + +limit) });
});
```

3. Body req.body (needs express.json() / express.urlencoded())

```
app.use(express.json());                                    // Content-Type: application/json
app.use(express.urlencoded({ extended: true }));            // Content-Type: application/x-www-form-urlencoded (HTML forms)
app.post("/api/login", (req, res) => {
  const { email, password } = req.body;                     // without the parser req.body is undefined!
  if (!email || !password) return res.status(400).json({ error: "email & password required" });
  res.json({ message: "Logged in", email });
});
```

req.body is undefined?

You forgot **app.use(express.json())** before the routes, or the client didn't send **Content-Type: application/json**.

Content types

JSON (APIs) · **urlencoded** (forms) · **multipart/form-data** (files → Multer) · **text/plain** (**express.text()**) · binary (**express.raw()**).

Never trust client data – validate every param, query and body field (Chapter 15).

6. Request & Response Objects in Detail

req – useful properties & methods

Property	Value / example
`req.method` / `req.url` / `req.originalUrl`	"GET" · "/users?x=1" (relative to mount) · full original URL
`req.path` / `req.baseUrl` / `req.route`	"/users" · router mount path "/api" · matched route info
`req.hostname` / `req.protocol` / `req.secure`	"api.site.com" · "http|https" · true if HTTPS
`req.ip` / `req.ips`	Client IP (use `trust proxy` behind Nginx/Cloudflare)
`req.get(h)` / `req.headers`	Header value / all headers
`req.is("json")` / `req.accepts("html")`	Check body type / content negotiation (what the client wants)
`req.xhr` / `req.fresh` / `req.cookies` / `req.signedCookies`	AJAX? / cache fresh? / cookies (needs cookie-parser)
`req.app` / `req.res` / `req.user`*	App instance · response · **custom** props added by middleware

res – sending responses

Method	What it does
`res.send(body)`	Send string / object / Buffer / array (auto Content-Type) – ends response
`res.json(obj)`	Send JSON (`Content-Type: application/json`) – most used in APIs
`res.status(code)`	Set status (chainable): `res.status(404).json(...)` · `res.sendStatus(204)`
`res.redirect([301|302], url)`	Redirect; `res.redirect("back")` removed in v5 → use `req.get("Referer")`
`res.render(view, data)`	Render a template (EJS/Pug) to HTML
`res.sendFile(absPath)` / `res.download(path)`	Send a file / force download (needs **absolute** path: `path.join(__dirname, ...)`)
`res.set(name, val)` / `res.type("json")`	Set header(s) / Content-Type
`res.cookie(n, v, opts)` / `res.clearCookie(n)`	Set / delete cookie
`res.format({"{ html(){}, json(){} }"})`	Reply depending on **Accept** header
`res.end()` / `res.location()` / `res.links()`	End without data · set Location header · Link headers
`res.locals` / `res.headersSent`	Per-request template vars · were headers already sent?

Examples

```
res.status(201).json({ id: 7, name: "Asad" });                    // JSON + status
res.send("<h1>Hi</h1>");   res.type("text/plain").send("plain text");
res.set({ "Cache-Control": "no-store", "X-Total-Count": 120 });          // headers
res.cookie("token", jwt, { httpOnly: true, secure: true, sameSite: "strict", maxAge: 3600_000 });
res.download(path.join(__dirname, "files", "report.pdf"), "my-report.pdf");
res.format({ "text/html": () => res.send("<p>hi</p>"), "application/json": () => res.json({ hi: 1 }), default: () => res.sendStatus(406) });
```

One response only

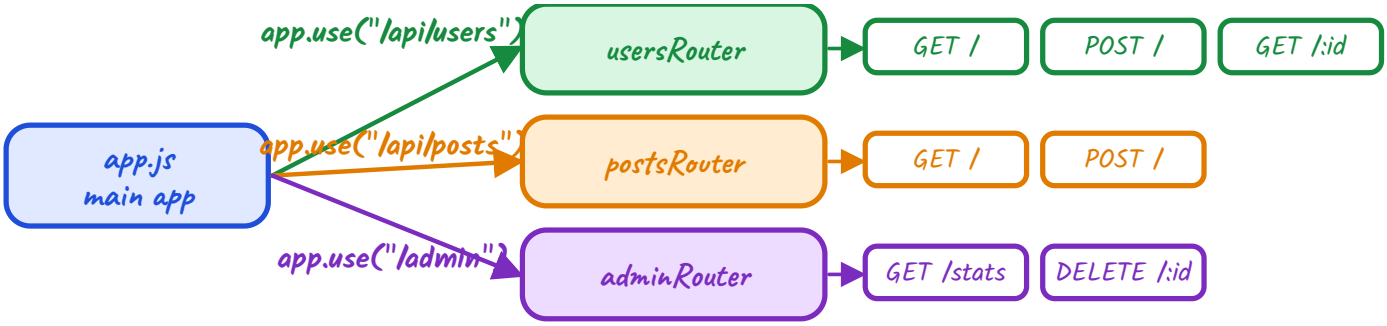
Only **one** of send/json/end/render/redirect per request. Write **`return res.status(400).json(...)`** in guards to stop execution.

Status code cheat

200 OK · 201 Created · 204 No Content · 301/302 Redirect · 400 Bad · 401 Login · 403 Forbidden · 404 Not found · 409 Conflict · 500 Server

7. express.Router – Modular Routes

A **Router** is a mini Express app that holds routes and middleware. Splitting routes into router files keeps big apps tidy.



Each router = a mini-app (own routes + middleware) mounted under a URL prefix

routes/users.js

```
import { Router } from "express";
const router = Router();

router.use((req, res, next) => { console.log("users router hit"); next(); });    // router-level middleware
router.get("/", (req, res) => res.json(users));                                   // GET  /api/users
router.get("/:id", (req, res) => res.json(users.find(u => u.id == req.params.id))); // GET  /api/users/5
router.post("/", (req, res) => res.status(201).json(req.body));                   // POST /api/users
router.route("/:id").put(updateUser).delete(deleteUser);
export default router;
```

app.js – mount routers with a prefix

```
import usersRouter from "./routes/users.js";   import postsRouter from "./routes/posts.js";
app.use("/api/users", usersRouter);            // all routes inside get the "/api/users" prefix
app.use("/api/posts", postsRouter);
app.use("/api/v1", v1Router);                  // easy API versioning: /api/v2 later
```

Nested routers & mergeParams

```
const postRouter = Router();
const commentRouter = Router({ mergeParams: true });      // lets the child read parent's :postId
commentRouter.get("/", (req, res) => res.json({ post: req.params.postId, comments: [] }));
postRouter.use("/:postId/comments", commentRouter);       // GET /posts/12/comments
app.use("/posts", postRouter);
```

Feature	Detail
router.use(fn)	Middleware for **every route** of this router (e.g. auth for all /admin routes)
router.param("id", fn)	Pre-process a param for this router only
req.baseUrl / req.originalUrl	Mount prefix ("/api/users") / full URL – paths inside a router are **relative**
Router options	**caseSensitive**, **strict**, **mergeParams**

Protect a whole router + 404 per router

```
const admin = Router();
admin.use(authenticate, requireRole("admin"));          // everything below needs an admin
admin.get("/stats", (req, res) => res.json({ users: 10 }));
admin.use((req, res) => res.status(404).json({ error: "Admin route not found" }));
app.use("/admin", admin);
```

Rule of thumb

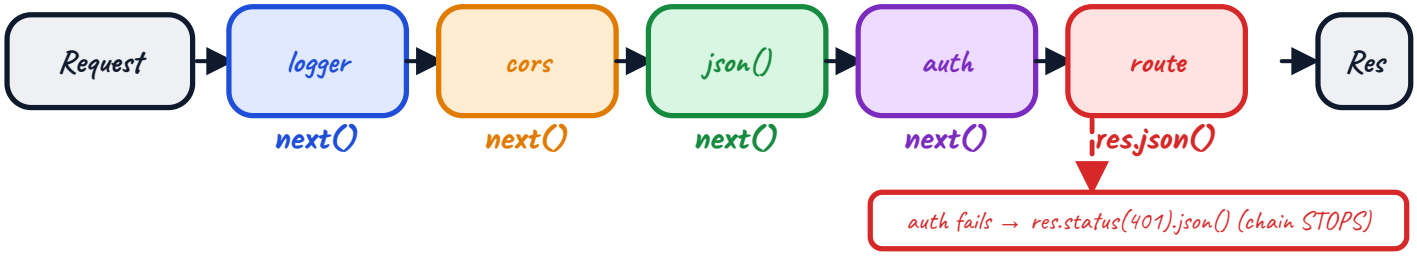
One router file per **resource** (users, posts, orders). The router only **maps URLs → controller functions**; logic lives in controllers (Chapter 20). Group routes by feature as the app grows.

8. Middleware – The Heart of Express

Middleware

A function with signature **(req, res, next)** that runs **between receiving the request and sending the response**. It can: execute code, **change req/res**, **end** the cycle, or call **next()** to pass control to the next function.

Every middleware gets (req, res, next) and decides: pass on, change, or end



Writing your own middleware

```
const logger = (req, res, next) => {                    // 1. logger
  const start = Date.now();
  res.on("finish", () => console.log(`${req.method} ${req.originalUrl} ${res.statusCode} ${Date.now() - start}ms`));
  next();                                               // IMPORTANT – otherwise the request hangs
};
const addTime = (req, res, next) => { req.requestTime = new Date().toISOString(); next(); };   // 2. attach data
const requireKey = (req, res, next) =>                                                          // 3. guard
  req.get("x-api-key") === process.env.API_KEY ? next() : res.status(401).json({ error: "Bad API key" });

app.use(logger);                          // runs for EVERY request (registered order = run order)
app.use(addTime);
app.use("/api", requireKey);              // only paths starting with /api
app.get("/api/data", requireKey, handler);    // only this route (route-level)
```

Level	How
Application-level	**app.use(fn) / app.use("/path", fn) / app.METHOD(path, fn)**
Router-level	**router.use(fn)** – same but bound to a Router
Built-in	**express.json(), express.static()** …
Third-party	morgan, cors, helmet, cookie-parser, multer …
Error-handling	**(err, req, res, next)** – 4 params

8. Middleware – next(), order & patterns

next() variations

Call	Effect
`next()`	Go to the **next middleware / handler**
`next("route")`	Skip rest of this route's handlers → next matching **route**
`next(err)`	Skip everything → **error-handling middleware**
`res.send(...)`	Ends the chain – later middleware is not run

Configurable middleware (factory) & async middleware

```
const requireRole = (...roles) => (req, res, next) =>        // function that RETURNS middleware
  roles.includes(req.user?.role) ? next() : res.status(403).json({ error: "Forbidden" });
app.delete("/users/:id", authenticate, requireRole("admin"), deleteUser);

const loadUser = async (req, res, next) => {                  // async middleware (Express 4: wrap try/catch + next(err))
  try { req.profile = await User.findById(req.user.id); next(); } catch (e) { next(e); }
};
```

Order matters!

Parsers (**express.json**) **before** routes; auth **before** protected routes; 404 handler **after** routes; error handler **last**. Typical order: helmet → cors → logger → parsers → routes → 404 → errors.

Mistakes

Forgetting **next()** (request hangs) · calling **next()** and sending a response · modifying **req** after the response was sent.

Watch the execution order (what prints?)

```
app.use((req, res, next) => { console.log("1 first");  next(); console.log("5 after next (runs on the way back)");
});
app.use((req, res, next) => { console.log("2 second"); next(); });
app.get("/test", (req, res, next) => { console.log("3 route A"); next(); }, (req, res) => { console.log("4 route B"); res.send("done"); });
// GET /test → 1 first, 2 second, 3 route A, 4 route B, 5 after next
```

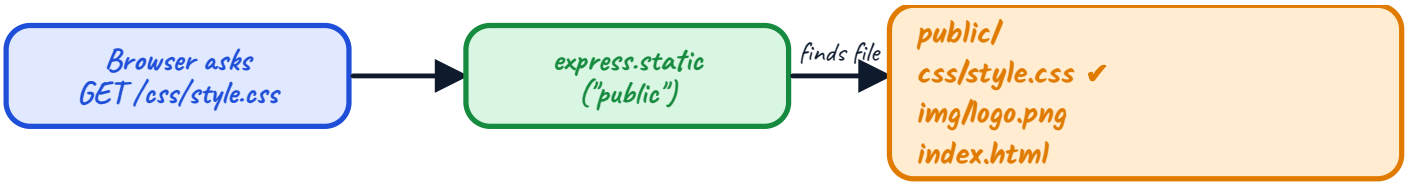
Handy middleware	Idea
Request ID	`req.id = crypto.randomUUID(); res.set("X-Request-Id", req.id)`
Response timer	`const t = process.hrtime.bigint(); res.on("finish", ...)`
Maintenance mode	`if (process.env.MAINTENANCE) return res.status(503).send("Back soon")`
Load resource	`router.param("id", async (req, res, next, id) => { req.book = await Book.findById(id); next(); })`

9. Built-in Middleware & Serving Static Files

Middleware	Purpose
express.json([options])	Parse **JSON** bodies → **req.body** (**limit: "1mb"**, **strict**, **type**)
express.urlencoded({"{ extended: true }"})	Parse **HTML form** bodies (**extended:true** = nested objects via qs library)
express.static(root, options)	Serve **static files** (HTML, CSS, JS, images, fonts)
express.text() / express.raw()	Plain text / **Buffer** bodies (webhooks, signatures)
express.Router()	Create modular routers (Chapter 7)

```
app.use(express.json({ limit: "10kb" }));                    // reject big payloads (413 Payload Too Large)
app.use(express.urlencoded({ extended: true }));
app.post("/webhook", express.raw({ type: "application/json" }), (req, res) => { /* req.body is a Buffer */
res.sendStatus(200); });
```

Serving static files



not found? → falls through to next middleware / routes → 404

```
import path from "node:path";   import { fileURLToPath } from "node:url";
const __dirname = path.dirname(fileURLToPath(import.meta.url));          // ESM replacement (or
import.meta.dirname in Node 20.11+)

app.use(express.static(path.join(__dirname, "public")));                  // /css/style.css → public/css/style.css
(use ABSOLUTE path)
app.use("/assets", express.static(path.join(__dirname, "public")));       // virtual prefix: /assets/css/style.css
app.use("/uploads", express.static("uploads"));                          // user uploads
app.use(express.static("public", {                                       // options
  maxAge: "7d",                 // Cache-Control max-age  (use hashed file names in production)
  etag: true, index: "index.html", extensions: ["html"],  dotfiles: "ignore",
}));
```

public/index.html

```
<link rel="stylesheet" href="/css/style.css">
<script src="/js/app.js"></script>
<img src="/img/logo.png" alt="logo">
```

- Requests for "/" return **index.html** automatically.
- Static middleware only handles **GET / HEAD**; if the file isn't found it calls **next()**.
- Put it **before** routes (fast) but after logging. Never serve a folder containing **.env** or source code.

Single Page App (React) fallback

Serve a built React/Vue app and let the client router handle URLs

```
app.use(express.static(path.join(__dirname, "client/dist")));
app.get("*splat", (req, res) => res.sendFile(path.join(__dirname, "client/dist/index.html")));    // Express 4:
app.get("*", ...)  — API routes must be defined BEFORE this
```

Production

Let **Nginx / a CDN** serve static files for speed; Express should mainly run the API. Enable **compression** and long **cache headers** for hashed assets.

10. Popular Third-party Middleware

Install with npm and plug in with **app.use()**. The most important ones:

Package	Purpose	Usage
morgan	HTTP request logger	app.use(morgan("dev"))
cors	Enable Cross-Origin requests	app.use(cors({" {origin:'http://localhost:5173'}"}))
helmet	Secure HTTP headers	app.use(helmet())
cookie-parser	Parse cookies → req.cookies	app.use(cookieParser(secret))
compression	gzip / brotli responses	app.use(compression())
express-rate-limit	Limit requests per IP	rateLimit({"{windowMs, limit}"})
multer	multipart / file uploads	upload.single("avatar")
express-session	Server-side sessions	session({"{secret, resave:false...}"})
method-override	PUT/DELETE from HTML forms	methodOverride("_method")
express-validator / zod	Input validation	body("email").isEmail()
http-proxy-middleware	Proxy to another server	createProxyMiddleware({" {target}"})
passport	Auth strategies (Google, GitHub…)	passport.authenticate("google")

A well-ordered production stack

```
import morgan from "morgan"; import cors from "cors"; import helmet from "helmet";
import compression from "compression"; import cookieParser from "cookie-parser"; import rateLimit from "express-rate-limit";

app.use(helmet());                                                    // 1. security headers first
app.use(cors({ origin: process.env.CLIENT_URL, credentials: true }));    // 2. who may call the API (credentials = cookies)
app.use(compression());                                               // 3. smaller responses
if (process.env.NODE_ENV !== "production") app.use(morgan("dev"));        // 4. logs  (GET /api/users 200 12.3 ms)
app.use(express.json({ limit: "10kb" })); app.use(express.urlencoded({ extended: true }));   // 5. parsers
app.use(cookieParser());
app.use("/api", rateLimit({ windowMs: 15 * 60 * 1000, limit: 100, standardHeaders: true, message: { error: "Too many requests" } }));
app.use("/api/users", usersRouter);                                   // 6. routes … then 404 + error handler
```

CORS explained

Browsers block a page on **localhost:5173** from calling an API on **localhost:3000** unless the API sends **Access-Control-Allow-Origin**. The **cors** package adds those headers (and answers **preflight OPTIONS**).

Morgan formats

"dev" colourful · **"combined"** Apache-style (production) · **"tiny"** · custom tokens. Write logs to a file stream or use **winston / pino**.

Choosing middleware: check weekly downloads, last update, open issues; prefer well-known packages; run **npm audit**. Each middleware adds latency → only use what you need. You can also publish **your own** middleware package.

11. Error Handling



4 parameters = Express knows it is an error handler · register it LAST

1. Sync & async errors

```
app.get("/sync", (req, res) => { throw new Error("Boom"); });          // sync throw → caught by Express automatically

app.get("/users/:id", async (req, res, next) => {                      // Express 4: async errors are NOT caught
  try {
    const user = await User.findById(req.params.id);
    if (!user) { const err = new Error("User not found"); err.status = 404; throw err; }
    res.json(user);
  } catch (err) { next(err); }                                          // forward to error middleware
});
// Express 5: rejected promises / thrown errors in async handlers go to the error handler AUTOMATICALLY
```

2. 404 handler + 3. Central error handler (both AFTER all routes)

```
app.use((req, res, next) => {                                          // reached only if no route matched
  res.status(404).json({ success: false, message: `Cannot ${req.method} ${req.originalUrl}` });
});
app.use((err, req, res, next) => {                                     // MUST have 4 params
  console.error(err);
  const status = err.status || err.statusCode || 500;
  res.status(status).json({
    success: false,
    message: status === 500 && process.env.NODE_ENV === "production" ? "Internal Server Error" : err.message,
    ...(process.env.NODE_ENV !== "production" && { stack: err.stack }),  // stack only in development
  });
});
```

11. Error Handling – custom errors & patterns

4. Custom error class + asyncHandler wrapper

```
export class AppError extends Error {
  constructor(message, status = 500) { super(message); this.status = status; this.isOperational = true; }
}
export const asyncHandler = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);   // Express 4 helper

app.get("/books/:id", asyncHandler(async (req, res) => {
  const book = await Book.findById(req.params.id);
  if (!book) throw new AppError("Book not found", 404);                    // clean controller code
  res.json(book);
}));
```

Error type	Typical handling
Validation (bad input)	400 / 422 with field messages
Auth	401 not logged in · 403 not allowed
Not found	404 · DB ***CastError*** (bad ObjectId) → 400/404
Duplicate key (Mongo 11000)	409 Conflict
Bugs / DB down	500, log details, hide message from client

Process-level safety net

```
process.on("unhandledRejection", (r) => { console.error("Unhandled:", r); server.close(() => process.exit(1)); });
process.on("uncaughtException", (e) => { console.error("Uncaught:", e); process.exit(1); });   // let PM2 / Docker restart the app
```

Rules

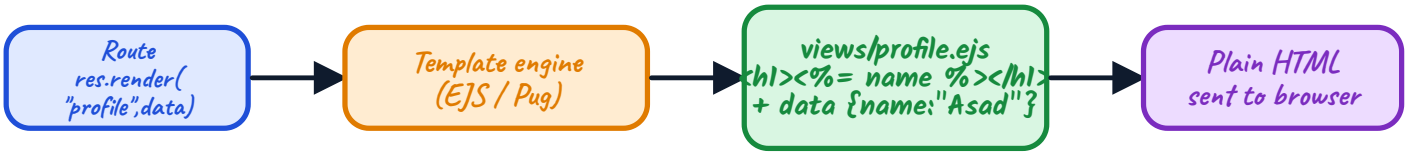
Error handler = **last** middleware · needs **4 args** even if ***next*** is unused · if headers were already sent call ***next(err)*** (default handler closes connection) · never send stack traces in production.

Map library errors to clean HTTP errors (inside the central handler)

```
app.use((err, req, res, next) => {
  if (err.name === "CastError") err = new AppError(`Invalid ${err.path}: ${err.value}`, 400);          // bad Mongo ObjectId
  if (err.code === 11000) err = new AppError(`Duplicate value: ${Object.keys(err.keyValue)}`, 409);     // unique index
  if (err.name === "ValidationError") err = new AppError(Object.values(err.errors).map(e => e.message).join(", "), 422);
  if (err.name === "JsonWebTokenError") err = new AppError("Invalid token", 401);
  if (err.name === "TokenExpiredError") err = new AppError("Token expired, log in again", 401);
  if (err.type === "entity.parse.failed") err = new AppError("Malformed JSON body", 400);               // body-parser error
  res.status(err.status || 500).json({ success: false, message: err.message });
});
```

12. Template Engines (Server-side Rendering)

A **template engine** fills an HTML template with data on the server and sends finished HTML. Good for classic websites, emails, SEO pages. (For React/Vue apps you usually build a JSON API instead.)



Engine	Syntax	Install
EJS	HTML + **<%= value %>** (easiest)	**npm i ejs**
Pug	Indentation-based, no closing tags	**npm i pug**
Handlebars	**{"{{value}}"}** logic-less	**npm i express-handlebars**
Nunjucks / Mustache	Jinja-style / logic-less	**npm i nunjucks**

Setup (EJS)

```
app.set("view engine", "ejs");                    // 1. choose engine (no import needed for ejs)
app.set("views", path.join(__dirname, "views"));  // 2. folder with templates (default ./views)
app.use(express.static("public"));                // css/js for the pages
app.locals.siteName = "My Blog";                  // global variable in ALL templates
app.get("/", (req, res) => res.render("index", { title: "Home", posts, user: req.user }));   // renders views/index.ejs
```

views/index.ejs

```
<%- include("partials/header", { title }) %>          <%# partial = reusable piece (header / footer / navbar) %>
<h1>Welcome to <%= siteName %></h1>                   <!-- <%= escapes HTML (safe) · <%- prints RAW HTML · <% runs JS -->
<% if (posts.length === 0) { %>
  <p>No posts yet.</p>
<% } else { %>
  <ul><% posts.forEach(post => { %>
    <li><a href="/posts/<%= post.id %>"><%= post.title %></a> – <%= post.date.toDateString() %></li>
  <% }) %></ul>
<% } %>
<%- include("partials/footer") %>
```

Pug equivalent

```
doctype html
html
  body
    h1 Welcome to #{siteName}
    if posts.length
      ul
        each post in posts
          li= post.title
    else
      p No posts yet.
```

- Layouts: **express-ejs-layouts** or Pug **extends / block**.
- **res.locals.user = req.user** in middleware → available in every view.
- Always use **escaped output** (**<%= %>**) to prevent **XSS**.
- Render to string (emails): **app.render("mail", data, cb)**.

Modern alternatives

React + Express API, **Next.js / Nuxt** (SSR frameworks), **HTMX** + partial templates. Template engines are still great for admin pages, emails and quick prototypes.

13. Forms, Cookies & Sessions

Handling HTML forms

form.html

```
<form action="/contact" method="POST">                    <!-- GET → data in URL (req.query) · POST → body (req.body) -->
  <input type="text" name="name" required>                 <!-- "name" attribute becomes the key in req.body -->
  <input type="email" name="email"><textarea name="message"></textarea>
  <button type="submit">Send</button>
</form>
```

```
app.use(express.urlencoded({ extended: true }));          // required for HTML forms
app.get("/contact", (req, res) => res.render("contact")); // show the form
app.post("/contact", (req, res) => {
  const { name, email, message } = req.body;
  // save / send email ...
  res.redirect("/thanks");                                // Post → Redirect → Get (no duplicate submit on refresh)
});
// HTML forms only support GET & POST → use method-override:  <form action="/books/5?_method=DELETE" method="POST">
app.use(methodOverride("_method"));   app.delete("/books/:id", deleteBook);
```

Cookies

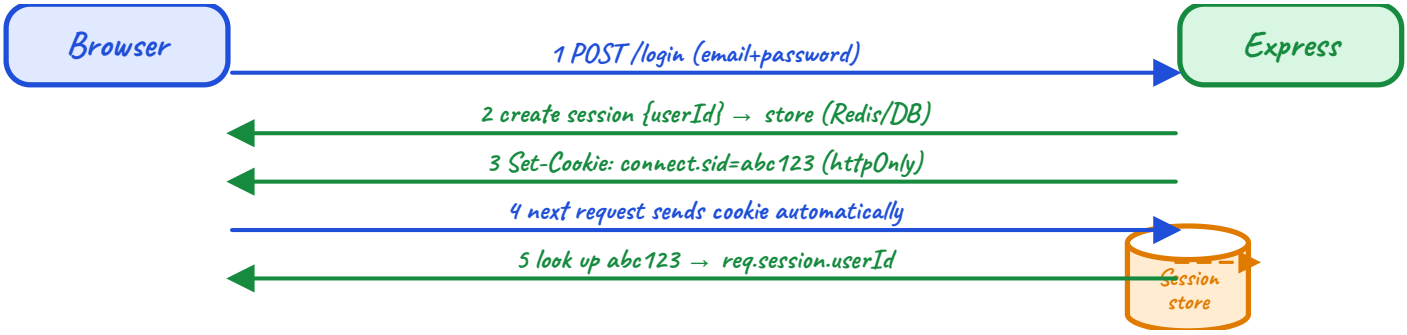
npm i cookie-parser

```
app.use(cookieParser("my-secret"));                       // secret enables signed cookies
app.get("/set", (req, res) => {
  res.cookie("theme", "dark", { maxAge: 7 * 24 * 3600 * 1000 });   // plain cookie (7 days)
  res.cookie("uid", "42", { httpOnly: true, secure: true, sameSite: "lax", signed: true });   // safe cookie
  res.send("cookies set");
});
app.get("/read", (req, res) => res.json({ plain: req.cookies, signed: req.signedCookies }));
app.get("/clear", (req, res) => { res.clearCookie("theme"); res.send("deleted"); });
```

Cookie option	Meaning
httpOnly	JavaScript in the browser **cannot read** it (blocks XSS theft)
secure	Sent only over **HTTPS**
sameSite	**"strict" | "lax" | "none"** - controls cross-site sending (CSRF defence)
maxAge / expires / domain / path	Lifetime and scope; **signed: true** detects tampering

13. Sessions & choosing a method

Sessions (express-session)



npm i express-session connect-mongo

```
import session from "express-session";   import MongoStore from "connect-mongo";
app.use(session({
  secret: process.env.SESSION_SECRET, resave: false, saveUninitialized: false,
  store: MongoStore.create({ mongoUrl: process.env.MONGO_URI }),          // MemoryStore is for development ONLY (leaks memory)
  cookie: { httpOnly: true, secure: process.env.NODE_ENV === "production", maxAge: 24 * 3600 * 1000 },
}));
app.post("/login", (req, res) => { /* verify password */ req.session.userId = user.id; res.json({ ok: true }); });
app.get("/me", (req, res) => req.session.userId ? res.json({ id: req.session.userId }) : res.status(401).end());
app.post("/logout", (req, res) => req.session.destroy(() => { res.clearCookie("connect.sid"); res.json({ ok: true }); }));
```

	Cookie	Session	JWT
Data stored	In the browser	On the server (browser keeps an id)	Inside the token (client)
Size / secrecy	~4 KB, visible to user	Large, private	Readable payload, signed
Scaling	Easy	Needs shared store (Redis) for many servers	Stateless - easy to scale
Logout / revoke	Delete cookie	Destroy session - instant	Hard until expiry (blacklist / short life)
Best for	Preferences (theme, language)	Server-rendered sites, simple login	APIs, mobile apps, microservices

Flash messages (one-time notices) & custom session data

```
// npm i connect-flash
app.use(flash());
app.post("/login", (req, res) => { req.flash("error", "Wrong password"); res.redirect("/login"); });
app.get("/login", (req, res) => res.render("login", { messages: req.flash("error") }));       // shown once, then removed
req.session.cart = [...(req.session.cart || []), item];                                       // any data per user: cart, language, step
req.session.regenerate(cb);   // new id after login → prevents session fixation
```

Session security

Strong **secret** from env · **httpOnly + secure + sameSite** cookies · **app.set("trust proxy", 1)** behind a proxy so secure cookies work · production store (Redis / Mongo / SQL) · regenerate id on login · set an idle timeout (**rolling: true**).

14. Building REST APIs & CRUD

Method	URL	Action	Success / errors
GET	**/api/v1/books**	List (filter, sort, page)	200
GET	**/api/v1/books/:id**	Read one	200 · 404
POST	**/api/v1/books**	Create	201 + Location · 400/422 · 409
PUT / PATCH	**/api/v1/books/:id**	Replace / update	200 · 404 · 422
DELETE	**/api/v1/books/:id**	Remove	204 · 404

controllers/book.controller.js (in-memory data to focus on Express)

```
let books = [{ id: 1, title: "Clean Code", author: "R. Martin", price: 30, year: 2008 }];
let nextId = 2;

export const listBooks = (req, res) => {
  let { q, author, minPrice, sort = "id", order = "asc", page = 1, limit = 10, fields } = req.query;
  let result = books.filter(b =>
    (!q || b.title.toLowerCase().includes(q.toLowerCase())) &&          // search
    (!author || b.author === author) && (!minPrice || b.price >= +minPrice));   // filter
  result.sort((a, b) => (a[sort] > b[sort] ? 1 : -1) * (order === "desc" ? -1 : 1));    // sort
  const total = result.length;
  result = result.slice((page - 1) * limit, page * limit);                              // paginate
  if (fields) result = result.map(b => Object.fromEntries(fields.split(",").map(f => [f, b[f]])));   // field
selection
  res.json({ page: +page, limit: +limit, total, pages: Math.ceil(total / limit), data: result });
};
export const getBook = (req, res) => {
  const book = books.find(b => b.id === +req.params.id);
  book ? res.json(book) : res.status(404).json({ error: "Book not found" });
};
export const createBook = (req, res) => {
  const book = { id: nextId++, ...req.body };  books.push(book);
  res.status(201).location(`/api/v1/books/${book.id}`).json(book);
};
export const updateBook = (req, res) => {                                  // PATCH style
  const i = books.findIndex(b => b.id === +req.params.id);
  if (i < 0) return res.status(404).json({ error: "Book not found" });
  books[i] = { ...books[i], ...req.body, id: books[i].id };  res.json(books[i]);
};
export const deleteBook = (req, res) => {
  const before = books.length;  books = books.filter(b => b.id !== +req.params.id);
  before === books.length ? res.status(404).json({ error: "Book not found" }) : res.status(204).end();
};
```

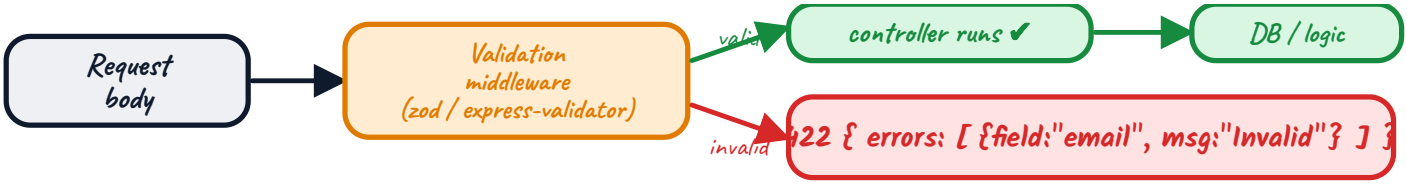
routes/book.routes.js

```
router.route("/").get(listBooks).post(validate(bookSchema), createBook);
router.route("/:id").get(getBook).patch(updateBook).delete(deleteBook);          // app.use("/api/v1/books",
router)
```

- **Nouns, plural**, lowercase, no verbs: **/books** not **/getBooks**.
- Nested: **/authors/3/books**. Version: **/api/v1**.
- Return **consistent JSON**: **{"{ success, data, message }"}**.
- **PUT** = replace (idempotent), **PATCH** = partial.
- Pagination: **?page=2&limit=10** (offset) or **?cursor=...** (large data).
- Sort **?sort=-price**, filter **?author=x**, search **?q=**.
- Document with **Swagger / OpenAPI** (**swagger-ui-express**).
- Rate-limit & cache GET responses (ETag / Cache-Control).

15. Input Validation & Sanitization

Never trust user input. Validate **type**, **format**, **length**, **range** and **sanitize** (clean) before using it. It prevents bugs, injection attacks and bad data.



Option 1 – express-validator

npm i express-validator

```
import { body, param, query, validationResult } from "express-validator";

const registerRules = [
  body("name").trim().notEmpty().withMessage("Name is required").isLength({ min: 2, max: 50 }),
  body("email").trim().isEmail().withMessage("Invalid email").normalizeEmail(),
  body("password").isLength({ min: 8 }).withMessage("Min 8 chars").matches(/\d/).withMessage("Needs a number"),
  body("age").optional().isInt({ min: 13, max: 120 }).toInt(),                    // sanitizer: convert to number
  body("confirm").custom((v, { req }) => v === req.body.password).withMessage("Passwords do not match"),
  param("id").isMongoId(),  query("page").optional().isInt({ min: 1 }).toInt(),
];
const validate = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) return res.status(422).json({ errors: errors.array().map(e => ({ field: e.path, message: e.msg })) });
  next();
};
app.post("/api/register", registerRules, validate, registerUser);
```

15. Validation with Zod

Option 2 – Zod schema (modern, great with TypeScript)

npm i zod

```
import { z } from "zod";
const userSchema = z.object({
  name: z.string().trim().min(2),
  email: z.string().email(),
  password: z.string().min(8, "Min 8 chars"),
  age: z.coerce.number().int().min(13).optional(),                    // "21" → 21
  role: z.enum(["user", "admin"]).default("user"),
});
export const validateBody = (schema) => (req, res, next) => {
  const result = schema.safeParse(req.body);
  if (!result.success) return res.status(422).json({ errors: result.error.flatten().fieldErrors });
  req.body = result.data;                    // cleaned + typed data
  next();
};
app.post("/api/users", validateBody(userSchema), createUser);
```

Library	Style
express-validator	Chain rules per field inside the route (wraps validator.js)
Zod	Schema object, infer TypeScript types, reusable on front-end
Joi / Yup / AJV	Schema libraries (Joi classic, AJV = JSON Schema, fastest)
Mongoose / DB constraints	Last line of defence (required, unique, enum, min/max)

Validate everywhere

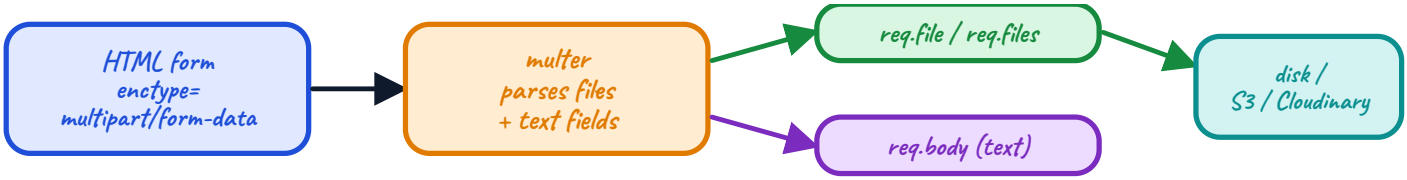
body + **params** + **query** + **headers**. Return 422 (or 400) with a clear list of field errors. Whitelist fields – never pass **req.body** straight into **User.create()** (mass assignment).

Sanitize

trim, **escape**, **normalizeEmail**, **toInt**; strip **$** / **.** keys for NoSQL injection (**express-mongo-sanitize**); client-side validation is only for UX.

16. File Upload with Multer

Files are sent as **multipart/form-data**, which ***express.json()*** cannot parse. **Multer** handles it: ***npm i multer***



upload.js – storage, limits & filter

```
import multer from "multer";   import path from "node:path";   import crypto from "node:crypto";
const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, "uploads/"),                                  // folder must exist
  filename: (req, file, cb) => cb(null, crypto.randomUUID() + path.extname(file.originalname)),   // never trust
original names
});
const fileFilter = (req, file, cb) =>
  ["image/png", "image/jpeg", "image/webp"].includes(file.mimetype) ? cb(null, true) : cb(new Error("Only images
allowed"), false);
export const upload = multer({ storage, fileFilter, limits: { fileSize: 2 * 1024 * 1024, files: 5 } });   // 2 MB
each
```

routes

```
app.post("/profile", upload.single("avatar"), (req, res) => {          // <input type="file" name="avatar">
  res.status(201).json({ file: req.file.filename, size: req.file.size, mimetype: req.file.mimetype, text: req.body.name
});
});
app.post("/gallery", upload.array("photos", 5), (req, res) => res.json({ count: req.files.length }));     // many,
same field
app.post("/mixed", upload.fields([{ name: "cover", maxCount: 1 }, { name: "docs", maxCount: 3 }]), (req, res) =>
res.json(req.files));
app.post("/form-only", upload.none(), (req, res) => res.json(req.body));                                  // text-only
multipart
app.use("/uploads", express.static("uploads"));                                                           // make files
viewable: /uploads/<name>
```

HTML: **<form method="POST" enctype="multipart/form-data">** with **<input type="file" name="avatar">** - **enctype is required**; the input name must equal the Multer field name.

Handle Multer errors + memory storage for cloud upload

```
app.use((err, req, res, next) => {
  if (err instanceof multer.MulterError) return res.status(400).json({ error: err.code });      //
LIMIT_FILE_SIZE, LIMIT_UNEXPECTED_FILE
  next(err);
});
const memUpload = multer({ storage: multer.memoryStorage() });          // file in req.file.buffer → send to S3
/ Cloudinary
app.post("/cloud", memUpload.single("img"), async (req, res) => { const url = await
uploadToCloudinary(req.file.buffer); res.json({ url }); });
```

req property	Content
req.file	{"{ fieldname, originalname, mimetype, size, path, filename, buffer }"}
req.files	Array (array) or object of arrays (fields)
req.body	Text fields sent in the same form

Security

Limit **size & count** · check **mimetype AND extension** (even better: magic bytes) · random file names · never execute uploads · store large/production files on **S3 / Cloudinary**.

17. Connecting Express to a Database



Each layer talks only to the next one – easy to test and replace (e.g. MongoDB → PostgreSQL)

MongoDB with Mongoose

config/db.js + server.js

```
import mongoose from "mongoose";
export default async function connectDB() {
  try { await mongoose.connect(process.env.MONGO_URI); console.log("MongoDB connected"); }
  catch (e) { console.error(e.message); process.exit(1); }                    // fail fast
}
// server.js:   await connectDB();  app.listen(PORT);        (start listening AFTER the DB is ready)
```

models/Book.js

```
const bookSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true },
  author: { type: String, required: true },
  price: { type: Number, min: 0, default: 0 },
  tags: [String],
}, { timestamps: true });
export default mongoose.model("Book", bookSchema);
```

controllers – async CRUD with proper status codes

```
export const getBooks = asyncHandler(async (req, res) => {
  const { page = 1, limit = 10, q } = req.query;
  const filter = q ? { title: new RegExp(q, "i") } : {};
  const [books, total] = await Promise.all([Book.find(filter).sort("-createdAt").skip((page - 1) *
limit).limit(+limit), Book.countDocuments(filter)]);
  res.json({ total, books });
});
export const createBook = asyncHandler(async (req, res) => res.status(201).json(await
Book.create(req.body)));
export const updateBook = asyncHandler(async (req, res) => {
  const book = await Book.findByIdAndUpdate(req.params.id, req.body, { new: true, runValidators: true });
  if (!book) throw new AppError("Book not found", 404);   res.json(book);
});
export const deleteBook = asyncHandler(async (req, res) => {
  if (!(await Book.findByIdAndDelete(req.params.id))) throw new AppError("Book not found", 404);
res.status(204).end();
});
```

17. SQL databases & ORMs

SQL (PostgreSQL / MySQL)

pg + Prisma

```
import pg from "pg";   const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL });        // connection pool
app.get("/users/:id", asyncHandler(async (req, res) => {
  const { rows } = await pool.query("SELECT id, name FROM users WHERE id = $1", [req.params.id]);        // $1 = parameter → no SQL injection
  rows[0] ? res.json(rows[0]) : res.status(404).json({ error: "Not found" });
}));
const user = await prisma.user.findUnique({ where: { id: +req.params.id }, include: { posts: true } });   // Prisma ORM
```

Best practices

- Connection string in **.env**
- One connection / pool for the whole app
- Put queries in **models / services**, not routes
- Use **indexes**, pagination, ***lean()***, projections

Never

- Concatenate user input into SQL
- Return password hashes (***select: false***)
- Pass raw ***req.body*** to ***create***
- Open a new DB connection per request

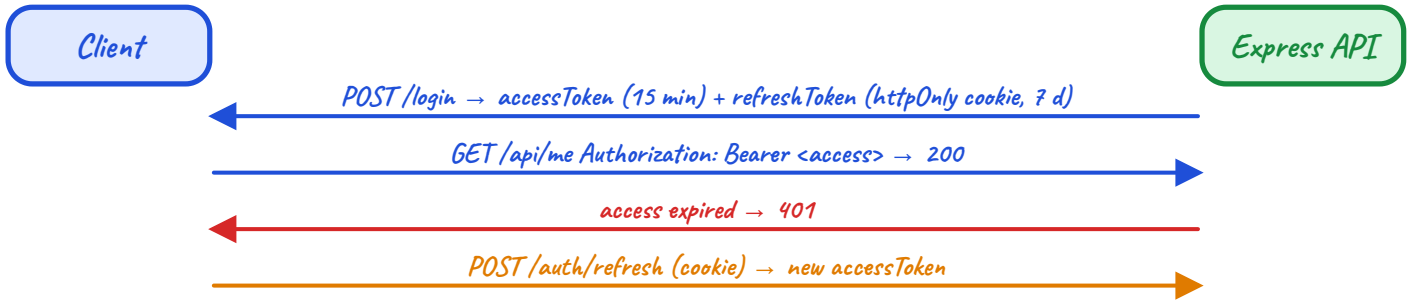
Prisma setup & a transaction

```
// npx prisma init  →  schema.prisma   model Book { id Int @id @default(autoincrement())  title String  author String  price Float @default(0) }
// npx prisma migrate dev --name init          // creates tables        npx prisma studio   // GUI
import { PrismaClient } from "@prisma/client";   export const prisma = new PrismaClient();
router.get("/", asyncHandler(async (req, res) => res.json(await prisma.book.findMany({ where: { price: { gte: +req.query.min || 0 } }, orderBy: { id: "desc" }, take: 10 }))));
await prisma.$transaction([ prisma.account.update({ where: { id: 1 }, data: { balance: { decrement: 50 } } }),
                            prisma.account.update({ where: { id: 2 }, data: { balance: { increment: 50 } } }) ]);
// both or none
```

Need	Choose
Flexible JSON documents	MongoDB + Mongoose
Relations, transactions, reports	PostgreSQL / MySQL + Prisma / Sequelize / Drizzle
Cache, sessions, rate limits	Redis (***ioredis***)
Tiny / local apps	SQLite (***better-sqlite3***)

18. Authentication & Authorization

Authentication = who are you? (login) · **Authorization** = what may you do? (roles / ownership). Two common approaches: **Sessions** (Chapter 13) and **JWT**.



controllers/auth.controller.js

```
import bcrypt from "bcrypt";  import jwt from "jsonwebtoken";
const signAccess = (u) => jwt.sign({ id: u._id, role: u.role }, process.env.JWT_SECRET, { expiresIn: "15m" });
const signRefresh = (u) => jwt.sign({ id: u._id }, process.env.JWT_REFRESH_SECRET, { expiresIn: "7d" });

export const register = asyncHandler(async (req, res) => {
  const { name, email, password } = req.body;
  if (await User.findOne({ email })) throw new AppError("Email already registered", 409);
  const user = await User.create({ name, email, password: await bcrypt.hash(password, 12) });        // hash, never store plain
  res.status(201).json({ id: user._id, email: user.email });
});
export const login = asyncHandler(async (req, res) => {
  const user = await User.findOne({ email: req.body.email }).select("+password");
  if (!user || !(await bcrypt.compare(req.body.password, user.password))) throw new AppError("Invalid email or password", 401);   // same message for both!
  res.cookie("refresh", signRefresh(user), { httpOnly: true, secure: true, sameSite: "strict", maxAge: 7 * 864e5 });
  res.json({ accessToken: signAccess(user) });
});
```

18. Auth middleware – protect, roles & ownership

middleware/auth.js – protect + restrictTo + ownership

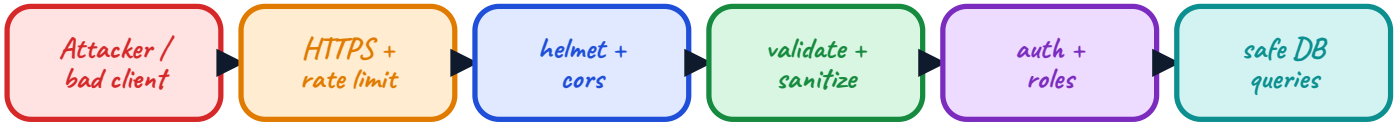
```
export const protect = asyncHandler(async (req, res, next) => {
  const token = req.headers.authorization?.startsWith("Bearer ") ? req.headers.authorization.split(" ")[1] :
req.cookies.token;
  if (!token) throw new AppError("Please log in", 401);
  const decoded = jwt.verify(token, process.env.JWT_SECRET);                    // throws
JsonWebTokenError / TokenExpiredError
  const user = await User.findById(decoded.id);
  if (!user) throw new AppError("User no longer exists", 401);
  req.user = user;  next();
});
export const restrictTo = (...roles) => (req, res, next) =>
  roles.includes(req.user.role) ? next() : next(new AppError("You do not have permission", 403));
export const isOwner = (Model) => asyncHandler(async (req, res, next) => {          // users may edit only
THEIR documents
  const doc = await Model.findById(req.params.id);
  if (!doc) throw new AppError("Not found", 404);
  if (doc.owner.toString() !== req.user.id && req.user.role !== "admin") throw new AppError("Forbidden", 403);
  req.doc = doc;  next();
});
```

use them

```
router.post("/register", validate(registerSchema), register);   router.post("/login", loginLimiter, login);
router.get("/me", protect, (req, res) => res.json(req.user));
router.delete("/:id", protect, restrictTo("admin"), deleteUser);   router.put("/posts/:id", protect, isOwner(Post),
updatePost);
```

- **bcrypt** cost 10–12 · JWT: short expiry + **refresh token** rotation · store refresh in **httpOnly cookie**.
- **RBAC** (roles) + **ownership** checks. 401 = not authenticated, 403 = not allowed.
- **Passport.js** strategies: local, Google / GitHub OAuth, JWT. Or services: **Auth0, Clerk, Firebase Auth.**
- Add email verification, password reset (token by mail), **2FA**, login rate-limit.

19. Security Best Practices



DEFENCE IN DEPTH – many small layers; an attacker must pass every one

Threat	Protection in Express
Unsafe headers	**helmet()** – CSP, HSTS, X-Frame-Options, noSniff; **app.disable("x-powered-by")**
CORS abuse	**cors({"{ origin: [\"https://myapp.com\"], credentials: true }"})** – whitelist, never * with cookies
Brute force / DoS	**express-rate-limit** (strict on /login), **express-slow-down**, body **limit**
Injection (NoSQL / SQL)	**express-mongo-sanitize**, parameterised queries / ORM, validate types (zod)
XSS	Escape output (**<%= %>**), sanitize HTML (**sanitize-html**), CSP, httpOnly cookies
CSRF	**sameSite** cookies + CSRF token (**csrf-csrf**) for cookie-based auth; JWT in header is immune
HTTP parameter pollution	**hpp()** – **?id=1&id=2** becomes an array!
Weak auth	bcrypt / argon2, strong JWT secrets, short expiry, lockout, 2FA
Sensitive data leaks	No stack traces in prod, **select: false** on passwords, **.env** out of git, redact logs
Vulnerable packages	**npm audit**, Dependabot / Renovate, lock file, update Node
Man-in-the-middle	HTTPS everywhere, HSTS, **secure** cookies

security setup

```
import helmet from "helmet"; import cors from "cors"; import rateLimit from "express-rate-limit";
import mongoSanitize from "express-mongo-sanitize"; import hpp from "hpp";
app.disable("x-powered-by"); app.set("trust proxy", 1); // real client IP behind Nginx / Heroku
app.use(helmet()); app.use(cors({ origin: process.env.CLIENT_URL, credentials: true }));
app.use(express.json({ limit: "10kb" })); app.use(mongoSanitize()); app.use(hpp());
app.use("/api", rateLimit({ windowMs: 15 * 60 * 1000, limit: 100 }));
app.use("/api/auth/login", rateLimit({ windowMs: 15 * 60 * 1000, limit: 5, message: { error: "Too many login attempts" } }));
```

Checklist

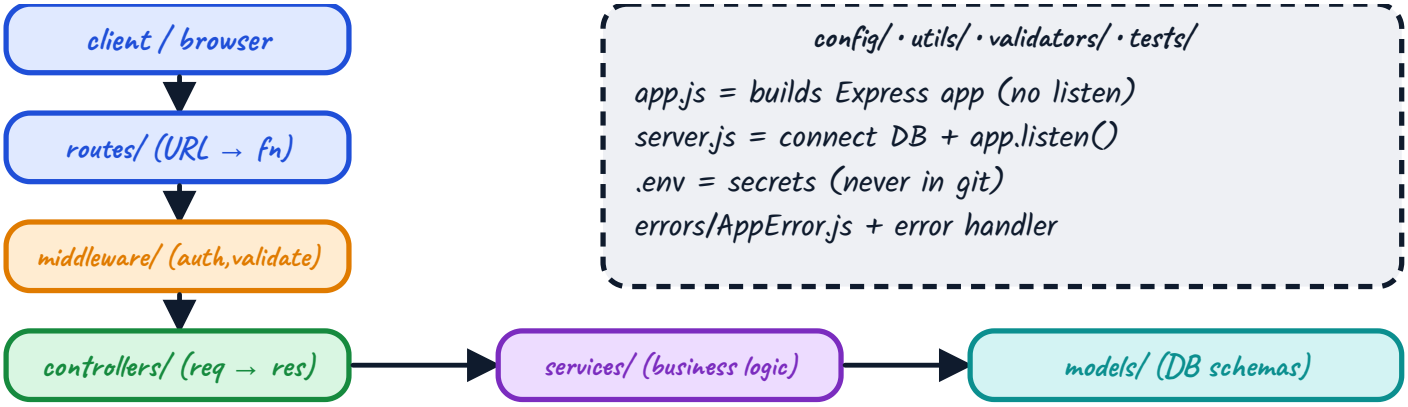
- Validate **all** input • hash passwords
- HTTPS + secure cookies • env secrets
- Least privilege for DB user & API keys
- Log security events, not passwords

Know

OWASP Top 10 (broken access control, injection, misconfiguration …). Test with **npm audit**, **OWASP ZAP**, Snyk.

20. Project Structure & MVC

MVC: Model (data & rules) · View (HTML template or JSON response) · Controller (handles request, calls model, sends view). Express doesn't force a structure – a clean one makes growth easy.



folder structure

```
my-api/
├── src/
│   ├── config/        db.js  env.js                  # connections &amp;amp; settings
│   ├── models/        User.js  Book.js               # Mongoose / Prisma
│   ├── controllers/   auth.controller.js  book.controller.js
│   ├── services/      book.service.js                # business logic &amp;amp; DB calls
│   ├── routes/        index.js  book.routes.js       # URL → controller
│   ├── middleware/    auth.js  validate.js  error.js
│   ├── validators/    book.schema.js                 # zod schemas
│   ├── utils/         AppError.js  asyncHandler.js
│   ├── app.js         # express() + middleware + routes (exported, testable)
│   └── server.js      # connectDB() then app.listen()
├── tests/   ├── .env  .env.example  .gitignore  ├── package.json  └── README.md
```

app.js / server.js / routes/index.js

```
const app = express();   app.use(helmet()); app.use(cors()); app.use(express.json());
app.use("/api/v1", routes);   app.use(notFound);   app.use(errorHandler);   export default app;      // app.js
// server.js:   await connectDB(); const server = app.listen(env.PORT, () => console.log(`API on ${env.PORT}`));
// routes/index.js:   router.use("/auth", authRoutes);  router.use("/books", bookRoutes);  router.get("/health", (req, res) => res.json({ status: "ok" }));
```

Layers in action: controller stays thin, service holds logic

```
// controller                                          // service
export const create = asyncHandler(async (req, res) => {      export const createBook = async (data, userId) => {
  const book = await bookService.createBook(req.body, req.user.id);      if (await Book.exists({ title: data.title })) throw new AppError("Exists", 409);
  res.status(201).json({ success: true, data: book });                  return Book.create({ ...data, owner: userId });
});                                                                   };
```

- Feature folders alternative: ***modules/books/{"{controller,service,routes,model}"}.js***.
- Central **config** reads ***process.env*** once and validates it (zod).
- Separate **app** from **server** so Supertest can import the app.
- Use **ESLint + Prettier**, path aliases, consistent naming.
- Keep a **README**, ***.env.example***, API docs (Swagger).
- Bigger apps: **NestJS** gives this structure by default.

21. App Settings & Advanced Topics

Application settings – app.set() / app.get()

Setting	Meaning
"view engine" / "views"	Template engine & folder
"trust proxy"	Trust X-Forwarded-* headers (1 = one proxy) → correct req.ip, req.protocol
"case sensitive routing" / "strict routing"	/Foo ≠ /foo · /foo ≠ /foo/
"etag"	ETag generation for caching ("weak" default, false to disable)
"json spaces" / "query parser"	Pretty JSON in dev · "simple" vs "extended" (Express 5 default: simple)
"x-powered-by"	app.disable("x-powered-by") hides Express
app.locals / res.locals	Variables for ALL templates / for THIS request

```
app.set("port", process.env.PORT || 3000);   app.get("port");   app.enable("trust proxy");   app.disabled("x-powered-by");
app.locals.appName = "Bookstore";   app.use((req, res, next) => { res.locals.user = req.user; next(); });
app.get("env");   // "development" (default) or NODE_ENV   if (app.get("env") === "production") { /* ... */ }
```

Server-Sent Events (SSE)



one open HTTP response, server keeps writing "data: ...\n\n" (live updates)

Proxy (http-proxy-middleware)



forward some paths to another server

Server-Sent Events – push live data (one-way)

```
app.get("/events", (req, res) => {
  res.set({ "Content-Type": "text/event-stream", "Cache-Control": "no-cache", Connection: "keep-alive" });
  res.flushHeaders();
  const timer = setInterval(() => res.write(`data: ${JSON.stringify({ time: Date.now() })}\n\n`), 1000);
  req.on("close", () => clearInterval(timer));                    // client left → stop
});   // browser: new EventSource("/events").onmessage = e => console.log(e.data)
```

Proxy, streaming, WebSockets, graceful shutdown

```
import { createProxyMiddleware } from "http-proxy-middleware";
app.use("/external", createProxyMiddleware({ target: "https://api.other.com", changeOrigin: true, pathRewrite: { "^/external": "" } }));
app.get("/video", (req, res) => fs.createReadStream("movie.mp4").pipe(res));                    // stream big files
const server = http.createServer(app);   const io = new Server(server);   server.listen(3000);   // Socket.IO shares the Express server
process.on("SIGTERM", () => server.close(() => { mongoose.connection.close(); process.exit(0); }));   // finish requests, then exit
```

Topic	Quick idea
Caching headers	res.set("Cache-Control", "public, max-age=3600"); ETag / 304 Not Modified
Compression	app.use(compression()) – gzip; let Nginx do it in prod
Content negotiation	req.accepts(["json","html"]) · res.format()
Subdomains / vhost	req.subdomains · vhost("api.site.com", apiApp)
Mounting sub-apps	const admin = express(); app.use("/admin", admin);
Background jobs	BullMQ + Redis, node-cron for scheduled tasks
HTTPS locally	https.createServer({"{key, cert}"}, app)

22. Logging & Debugging

Logging requests with morgan

```
import morgan from "morgan";   import fs from "node:fs";
app.use(morgan("dev"));                                                    // :method :url :status :response-time ms (colourful, development)
app.use(morgan("combined", { stream: fs.createWriteStream("access.log", { flags: "a" }) }));   // Apache format → file (production)
morgan.token("user", (req) => req.user?.id ?? "-");                        // custom token
app.use(morgan(":method :url :status :res[content-length] - :response-time ms user=:user"));
```

Application logging with pino / winston

npm i pino pino-http (fast JSON logs)

```
import pino from "pino";   import pinoHttp from "pino-http";
export const logger = pino({ level: process.env.LOG_LEVEL || "info", redact: ["req.headers.authorization", "password"] });   // hide secrets
app.use(pinoHttp({ logger }));                                             // logs every request with id, status, time
app.get("/pay", (req, res) => { req.log.info({ orderId: 5 }, "payment started"); res.sendStatus(200); });
logger.error({ err }, "DB connection failed");                             // error with stack
```

Level	Use
error	Something failed (exceptions, 5xx)
warn	Unexpected but handled (retry, 4xx spikes)
info	Normal events (server started, order created)
debug	Detailed developer info (off in production)

- Production: **JSON logs** to stdout → collected by Docker / cloud (Datadog, ELK, Loki). Add a **request id** to trace one request. **Never log** passwords, tokens, card numbers.
- Error tracking: **Sentry** (***Sentry.setupExpressErrorHandler(app)***). Uptime: UptimeRobot + ***/health*** route.

Debugging

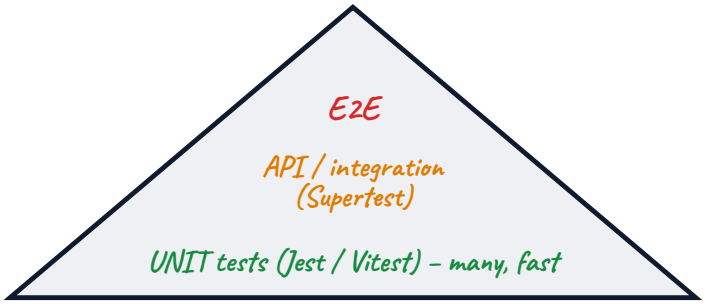
```
node --inspect app.js          # then open chrome://inspect  (or VS Code → Run and Debug → "Node.js")
DEBUG=express:* node app.js    # internal Express logs (routing, middleware)    DEBUG=express:router for routes only
node --watch app.js            # restart on change
```

Symptom	Likely cause & fix
Cannot GET /path	No route / wrong method / router prefix; check order & mount path
req.body is undefined	Missing ***express.json()*** or wrong Content-Type; parser registered after routes
Request hangs forever	Handler never sent a response or never called ***next()***
ERR_HTTP_HEADERS_SENT	Two responses (missing ***return*** before res.send)
CORS error in browser	Add ***cors()*** with correct origin; handle OPTIONS preflight; credentials
EADDRINUSE :3000	Port busy: kill old process (***lsof -i :3000***) or change PORT
Cookie not saved	***secure*** on HTTP, no ***credentials: true***, ***sameSite***, missing ***trust proxy***

Tools

VS Code breakpoints + debug console · **Postman** console · ***console.table*** · **express-list-endpoints** to print all routes · **nodemon** / ***--watch***.

23. Testing Express Apps



Supertest sends real HTTP requests to your ***app*** object **without starting a port** – that's why we export ***app*** separately from ***server.js***.

```
npm i -D jest supertest          # or vitest.   package.json: "test": "jest"   (ESM: NODE_OPTIONS=--experimental-vm-modules jest)
npm test      npx jest --watch      npx jest --coverage
```

tests/books.test.js

```
import request from "supertest";   import app from "../src/app.js";   import mongoose from "mongoose";

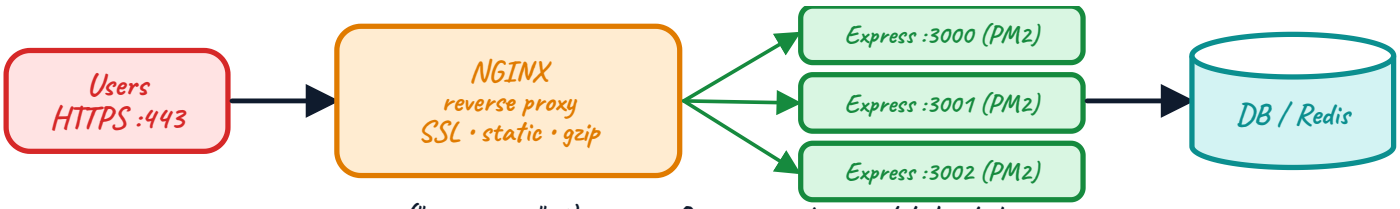
beforeAll(async () => await mongoose.connect(process.env.MONGO_TEST_URI));      // separate TEST database!
afterEach(async () => await Book.deleteMany({}));                                // clean state between tests
afterAll(async () => await mongoose.disconnect());

describe("GET /api/v1/books", () => {
  it("returns an empty list", async () => {
    const res = await request(app).get("/api/v1/books").expect("Content-Type", /json/).expect(200);
    expect(res.body.data).toEqual([]);
  });
});
describe("POST /api/v1/books", () => {
  it("creates a book", async () => {
    const res = await request(app).post("/api/v1/books").send({ title: "Node", author: "Asad", price: 10 });
    expect(res.status).toBe(201);   expect(res.body).toMatchObject({ title: "Node" });
expect(res.body._id).toBeDefined();
  });
  it("rejects invalid data", async () => { await request(app).post("/api/v1/books").send({}).expect(422); });
});
describe("protected route", () => {
  it("401 without token", async () => { await request(app).get("/api/v1/me").expect(401); });
  it("200 with token", async () => {
    const { body } = await request(app).post("/api/v1/auth/login").send({ email: "a@x.com", password: "secret123" });
    await request(app).get("/api/v1/me").set("Authorization", `Bearer ${body.accessToken}`).expect(200);
  });
});
```

Tool / concept	Use
request(app).get/post/put/delete	Call routes; chain **.send(json)**, **.set(header)**, **.attach("file", path)**, **.expect(status)**
jest.fn() / jest.mock("./service")	Mock functions / modules (email, payments, external APIs) – unit-test controllers in isolation
mongodb-memory-server	In-memory MongoDB for fast tests (no real DB)
Arrange – Act – Assert	Set up data → call endpoint → check status/body/DB

Test **success, validation errors, 401/403, 404**; keep tests independent; run them in **CI**. Also: ***node:test*** (built-in), Newman, Playwright E2E.

24. Performance & Production Best Practices



app.set("trust proxy", 1) so req.ip & secure cookies work behind the proxy

Area	Practice
NODE_ENV=production	Express caches views, uses less verbose errors – up to **3× faster**. Set it on the server!
Async only	No ***Sync** functions (readFileSync, bcrypt.hashSync) in request handlers – they block every user
Compression & caching	**compression()** or Nginx gzip/brotli • **Cache-Control**/ETag • **Redis** for hot data
Database	Indexes, pagination, projections, **lean()**, connection pooling, avoid N+1 queries
Cluster / PM2	Use all CPU cores: **pm2 start src/server.js -i max** (or cluster module / containers)
Reverse proxy	Nginx / cloud load balancer for HTTPS, static files, rate limits, buffering slow clients
Reliability	**/health** endpoint, auto-restart (PM2/Docker), graceful shutdown (SIGTERM), timeouts
Observability	Structured logs, metrics (Prometheus), tracing, Sentry, uptime checks

Response caching with Redis + gzip + timeout

```
const cacheMw = (ttl = 60) => async (req, res, next) => {
  const key = "cache:" + req.originalUrl;   const hit = await redis.get(key);
  if (hit) return res.set("X-Cache", "HIT").json(JSON.parse(hit));
  const send = res.json.bind(res);   res.json = (body) => { redis.setEx(key, ttl, JSON.stringify(body)); return
send(body); };   next();
};
app.get("/api/v1/products", cacheMw(120), listProducts);        // invalidate with redis.del(key) after
POST/PUT/DELETE
app.use(compression());   server.setTimeout(30_000);   server.keepAliveTimeout = 65_000;      // above load-
balancer idle timeout
```

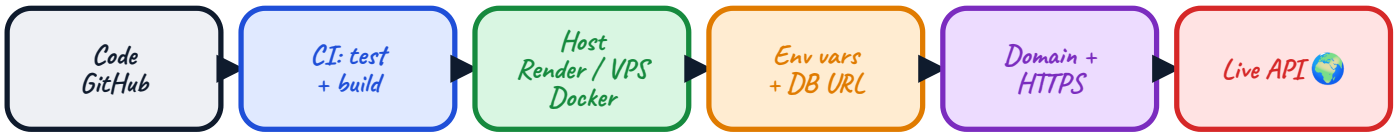
ecosystem.config.cjs (PM2)

```
module.exports = { apps: [{ name: "api", script: "src/server.js", instances: "max", exec_mode: "cluster",
env_production: { NODE_ENV: "production", PORT: 3000 }, max_memory_restart: "300M" }] };
// pm2 start ecosystem.config.cjs --env production  •  pm2 logs  •  pm2 reload api (zero downtime)  •
pm2 save &amp;amp;&amp;amp; pm2 startup
```

Quick wins

Cache what is read often • paginate everything • avoid huge JSON • stream large files • keep payloads small • measure first with **autocannon / k6 / clinic.js** – optimise the slowest part, not guesses.

25. Deploying an Express App



Prepare for production

- → **"start": "node src/server.js"** in package.json
- → Listen on **process.env.PORT** (host assigns it)
- → Env vars on the host (never upload .env)
- → **NODE_ENV=production**, **engines.node**
- → Cloud DB: MongoDB Atlas / Neon / Supabase
- → CORS origin = real front-end URL
- → helmet, rate-limit, **trust proxy**, **/health**
- → .gitignore node_modules + .env; commit lock file

Platform	How
Render / Railway / Fly.io	Connect GitHub → build **npm install**, start **npm start** → auto-deploy on push (beginner-friendly, free tiers)
Vercel / Netlify	Serverless functions: export the app as a handler (**export default app** + vercel.json rewrite)
Heroku / Azure / Google Cloud Run	PaaS / managed containers; Procfile **web: node src/server.js**
VPS (EC2, DigitalOcean)	Linux + Node + **PM2** + **Nginx** + Let's Encrypt HTTPS - full control
Docker / Kubernetes	Same image everywhere (ECS, Cloud Run, AKS)

Dockerfile + .dockerignore (node_modules, .env, .git)

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev                  # cached layer: only reruns when deps change
COPY . .
ENV NODE_ENV=production
EXPOSE 3000
USER node                              # don't run as root
CMD ["node", "src/server.js"]
```

VPS: PM2 + Nginx + HTTPS

```
git clone <repo> && cd app && npm ci --omit=dev && pm2 start src/server.js -i max --name api && pm2 save && pm2 startup
# /etc/nginx/sites-available/api →   location / { proxy_pass http://127.0.0.1:3000; proxy_set_header Host $host; proxy_set_header X-Forwarded-For $remote_addr; proxy_set_header X-Forwarded-Proto $scheme; }
sudo nginx -t && sudo systemctl reload nginx        sudo certbot --nginx -d api.example.com        # free SSL, auto-renew
```

.github/workflows/deploy.yml - CI/CD

```
on: { push: { branches: [main] } }
jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci && npm test
      - run: curl -X POST ${{ secrets.RENDER_DEPLOY_HOOK }}      # trigger deploy only if tests passed
```

After going live

Check logs, **/health**, HTTPS, CORS from the real front-end; set up monitoring (UptimeRobot, Sentry), DB backups, and a rollback plan (previous image / git tag).

26. Express 5 & TypeScript

What changed in Express 5 (npm i express@5 – now the default)

Express 4

async errors → need try/catch + next(err) or asyncHandler
"*" wildcard · app.del()
body-parser separate

→

Express 5

async errors → handled automatically
"*splat" named wildcard
req.query read-only getter
Node 18+ · res.json(status,obj) removed

Change	Details / migration
Async errors auto-forwarded	A rejected promise or throw in a handler calls **next(err)** – no more **asyncHandler** wrapper
Path syntax (path-to-regexp v8)	"*" → "/*splat"; optional "/:id?" → "/{"{/:id}"}"; no inline regex in params; "*" alone is invalid
Removed methods	**app.del() → app.delete() · res.json(status, obj) → res.status(s).json(obj) · res.redirect("back") · req.param()**
req.body	**undefined** (not {"{}"}) if no parser ran
req.query / req.host	Query is a getter (don't reassign) · **req.host** includes port · default query parser "simple"
Requires Node 18+	Run the codemod: **npx @expressjs/codemod upgrade**

Express with TypeScript

```
npm i express && npm i -D typescript tsx @types/express @types/node        # tsx runs TS directly
npx tsc --init          # tsconfig: "target":"ES2022", "module":"NodeNext", "outDir":"dist", "strict":true
# package.json →   "dev": "tsx watch src/server.ts"   "build": "tsc"   "start": "node dist/server.js"
```

src/server.ts

```ts
import express, { Request, Response, NextFunction } from "express";
const app = express();   app.use(express.json());

interface User { id: number; name: string; email: string }
const users: User[] = [];

app.post("/users", (req: Request<{}, {}, Omit<User, "id">>, res: Response<User>) => {      // Request<Params, ResBody, ReqBody, Query>
  const user: User = { id: users.length + 1, ...req.body };   users.push(user);
  res.status(201).json(user);
});
app.get("/users/:id", (req: Request<{ id: string }>, res: Response<User | { error: string }>) => {
  const user = users.find(u => u.id === Number(req.params.id));
  user ? res.json(user) : res.status(404).json({ error: "Not found" });
});
app.use((err: Error, req: Request, res: Response, next: NextFunction) => { res.status(500).json({ error: err.message }); });
app.listen(3000);
```

Why TypeScript?

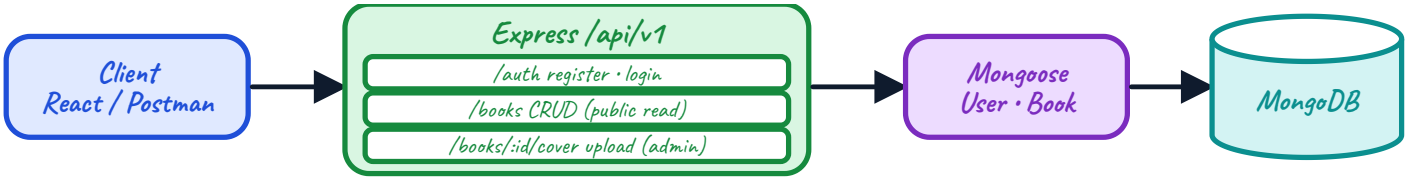
Catches wrong property names / types at compile time, better autocomplete, safer refactors. Combine with **Zod**: ***type Body = z.infer<typeof schema>***.

Next steps

NestJS (structured TS framework on Express), **Fastify**, **tRPC**, **GraphQL** (Apollo Server ***expressMiddleware***), **Next.js API routes**.

27. Mini Project – Bookstore API

Features: register / login (JWT) · public book listing with search, filter, sort & pagination · admin-only create / update / delete · cover image upload · validation · central error handling · tests. **Stack:** Express, Mongoose, zod, bcrypt, jsonwebtoken, multer, helmet, cors, morgan.



Setup

```
mkdir bookstore && cd bookstore && npm init -y && npm pkg set type=module
npm i express mongoose zod bcrypt jsonwebtoken multer helmet cors morgan dotenv && npm i -D nodemon jest supertest
mkdir -p src/{config,models,controllers,routes,middleware,utils} uploads      # .env → PORT MONGO_URI JWT_SECRET CLIENT_URL
```

src/app.js

```
import express from "express";  import helmet from "helmet";  import cors from "cors";  import morgan from "morgan";
import authRoutes from "./routes/auth.routes.js";  import bookRoutes from "./routes/book.routes.js";
import { notFound, errorHandler } from "./middleware/error.js";

const app = express();
app.use(helmet());  app.use(cors({ origin: process.env.CLIENT_URL }));  app.use(morgan("dev"));
app.use(express.json({ limit: "10kb" }));   app.use("/uploads", express.static("uploads"));
app.get("/api/v1/health", (req, res) => res.json({ status: "ok", uptime: process.uptime() }));
app.use("/api/v1/auth", authRoutes);   app.use("/api/v1/books", bookRoutes);
app.use(notFound);   app.use(errorHandler);
export default app;
```

models/Book.js

```
const bookSchema = new Schema({
  title: { type: String, required: true, trim: true },
  author: { type: String, required: true },
  price: { type: Number, min: 0, required: true },
  genre: { type: String, enum: ["tech","fiction","history","other"], default: "other" },
  cover: String,
  createdBy: { type: Schema.Types.ObjectId, ref: "User" },
}, { timestamps: true });
bookSchema.index({ title: "text", author: "text" });
export default model("Book", bookSchema);
```

middleware/error.js

```
export const notFound = (req, res, next) =>
  next(new AppError(`Cannot ${req.method} ${req.originalUrl}`, 404));
export const errorHandler = (err, req, res, next) => {
  if (err.name === "ValidationError") err.status = 422;
  if (err.code === 11000) err.status = 409;
  res.status(err.status || 500).json({
    success: false, message: err.message,
    ...(process.env.NODE_ENV === "development" && { stack: err.stack }),
  });
};
```

27. Bookstore API – controllers, routes & test

controllers/book.controller.js

```
export const getBooks = asyncHandler(async (req, res) => {
  const { q, genre, minPrice, maxPrice, sort = "-createdAt", page = 1, limit = 10 } = req.query;
  const filter = {};
  if (q) filter.$text = { $search: q };                                    // full-text search
  if (genre) filter.genre = genre;
  if (minPrice || maxPrice) filter.price = { ...(minPrice && { $gte: +minPrice }), ...(maxPrice && { $lte:
+maxPrice }) };
  const [data, total] = await Promise.all([
    Book.find(filter).sort(sort.split(",").join(" ")).skip((page - 1) * limit).limit(+limit),
Book.countDocuments(filter)]);
  res.json({ success: true, page: +page, pages: Math.ceil(total / limit), total, data });
});
export const createBook = asyncHandler(async (req, res) => res.status(201).json(await Book.create({
...req.body, createdBy: req.user.id })));
export const updateBook = asyncHandler(async (req, res) => {
  const book = await Book.findByIdAndUpdate(req.params.id, req.body, { new: true, runValidators: true });
  if (!book) throw new AppError("Book not found", 404);   res.json(book);
});
export const deleteBook = asyncHandler(async (req, res) => {
  if (!(await Book.findByIdAndDelete(req.params.id))) throw new AppError("Book not found", 404);
res.status(204).end();
});
export const uploadCover = asyncHandler(async (req, res) => {
  if (!req.file) throw new AppError("Image required", 400);
  res.json(await Book.findByIdAndUpdate(req.params.id, { cover: `/uploads/${req.file.filename}` }, { new: true }));
});
```

routes/book.routes.js

```
const router = Router();
router.get("/", getBooks);   router.get("/:id", getBook);                  // public
router.use(protect, restrictTo("admin"));                                  // everything below:
admin only
router.post("/", validateBody(bookSchema), createBook);
router.route("/:id").patch(validateBody(bookSchema.partial()), updateBook).delete(deleteBook);
router.post("/:id/cover", upload.single("cover"), uploadCover);
export default router;
```

Try it

```
curl "localhost:3000/api/v1/books?q=node&genre=tech&sort=-price&page=1&limit=5"
curl -X POST localhost:3000/api/v1/books -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json"
-d '{"title":"Express Guide","author":"Asad","price":19}'
curl -X POST localhost:3000/api/v1/books/ID/cover -H "Authorization: Bearer $TOKEN" -F "cover=@./cover.png"
```

tests/books.test.js

```
it("non-admin cannot create a book", async () => {
  await request(app).post("/api/v1/books").set("Authorization", `Bearer
${userToken}`).send(validBook).expect(403);
});
it("lists books with pagination", async () => {
  const res = await request(app).get("/api/v1/books?limit=2").expect(200);
expect(res.body.data.length).toBeLessThanOrEqual(2);
});
```

Extend it

Reviews & ratings (nested routes) · cart & orders · Stripe payments · email verification (Nodemailer) · Redis caching · Swagger docs · Docker + GitHub Actions · React front-end · deploy on Render with MongoDB Atlas.

28. Express.js Interview Questions & Answers

1. **What is Express.js?** A minimal, flexible Node.js web framework providing routing, middleware and HTTP helpers for building web apps and APIs.
2. **Why use Express instead of the http module?** Less boilerplate: routing, body parsing, static files, middleware chain, error handling, templating.
3. **Is Express a framework or library? Opinionated?** A (minimal) web framework; **unopinionated** – you choose structure, DB and tools.
4. **What is middleware?** A function ***(req, res, next)*** that runs during the request cycle; can modify req/res, end the response, or call next().
5. **Types of middleware?** Application-level, router-level, built-in, third-party, error-handling.
6. **What does *next()* do? *next(err)*?** Passes control to the next handler; with an argument it skips to the error-handling middleware.
7. **Does middleware order matter?** Yes – executed in registration order. Parsers/auth before routes; 404 after routes; error handler last.
8. **How do you handle errors?** try/catch + ***next(err)*** (Express 4) or auto in Express 5; central ***(err,req,res,next)*** handler; 404 handler.
9. **How does Express know a function is an error handler?** It has **four** parameters.
10. **req.params vs req.query vs req.body?** Path variables ***/:id*** • URL query ***?a=1*** • request payload (needs express.json()).
11. **What is express.json()?** Built-in middleware parsing JSON bodies into ***req.body***. ***urlencoded*** does the same for HTML forms.
12. **What is a Router?** ***express.Router()*** – a mini app with its own routes/middleware, mounted with ***app.use("/prefix", router)***.
13. **app.use vs app.get?** use matches any method and path prefix (middleware/routers); get matches only GET and the exact path.
14. **res.send vs res.json vs res.end?** send: auto type (string/object/buffer); json: JSON with header; end: no data, just finishes.
15. **How to serve static files?** ***app.use(express.static("public"))***; optional virtual prefix and cache options.
16. **What is a template engine?** Renders HTML with data on the server (EJS, Pug, Handlebars) via ***res.render()***.
17. **PUT vs PATCH vs POST?** Replace / partial update / create.
18. **Which status codes do you use?** 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500.
19. **What is CORS and how to enable it?** Browser same-origin policy relaxation; use the ***cors*** package with a whitelist (handles preflight OPTIONS).
20. **How to start the server?** ***app.listen(PORT, cb)*** – wraps ***http.createServer(app)***.

28. Express Interview Q&A (continued)

1. **How do you structure a large Express app?** routes → controllers → services → models, plus middleware, config, utils; app.js separate from server.js.
2. **How to secure an Express app?** helmet, cors, rate-limit, validation/sanitization, hashed passwords, JWT/session hardening, HTTPS, env secrets, npm audit.
3. **Sessions vs JWT?** Server-stored state + cookie id vs stateless signed token sent by client; sessions easy to revoke, JWT easy to scale.
4. **How to implement authentication?** Hash with bcrypt, verify at login, issue JWT/session, ***protect*** middleware verifies and sets ***req.user***, ***restrictTo*** for roles.
5. **How to upload files?** Multer middleware (***single / array / fields***), limits + filter, store on disk/S3, form needs ***multipart/form-data***.
6. **How to validate input?** express-validator or Zod/Joi middleware returning 422 with messages; whitelist fields.
7. **What is *app.locals* vs *res.locals*?** Template variables for the whole app vs for the current request.
8. **What is *trust proxy*?** Tells Express to trust X-Forwarded-* headers from a reverse proxy so ***req.ip/protocol*** and secure cookies work.
9. **How to handle async errors in Express 4?** try/catch with next(err) or a wrapper ***fn(req,res,next).catch(next)***; Express 5 does it automatically.
10. **How to avoid "headers already sent"?** Send exactly one response; use ***return*** after res.send in guards.
11. **How do you test Express routes?** Jest/Vitest + Supertest against the exported app, test DB, mocks for external services.
12. **How to improve performance?** compression, caching (Redis/ETag), indexes + pagination, async code, cluster/PM2, Nginx, NODE_ENV=production.
13. **How to handle CORS preflight?** Browser sends OPTIONS first; cors middleware answers with allowed methods/headers.
14. **What is the difference between *app.route()* and router?** route chains handlers for one path; router groups many paths.
15. **How do you version an API?** URL prefix ***/api/v1*** with separate routers (or Accept header).
16. **How to deploy?** PORT from env, ***npm start***, host on Render/VPS/Docker, PM2 + Nginx + HTTPS, CI/CD.
17. **What changed in Express 5?** Async errors handled, new path syntax (****splat***), removed deprecated methods, requires Node 18+.
18. **Express vs Fastify vs NestJS?** Minimal & popular / faster with schemas / opinionated TypeScript framework.
19. **Explain the request lifecycle.** Request → global middleware → router match → route middleware → controller → response (or error → error handler).

Interview tip

Walk through a flow: "A request hits helmet/cors/json parser, my router matches ***/books***, ***protect*** checks the JWT, zod validates, the controller calls the service, Mongoose saves, and any error goes to the central handler."

29. Express.js Cheat Sheet

Setup

```
npm init -y && npm pkg set type=module
npm i express && npm i -D nodemon
node --watch app.js
```

Basic app

```
import express from "express";
const app = express();
app.use(express.json());
app.use(express.urlencoded({extended:true}));
app.use(express.static("public"));
app.get("/", (req, res) => res.send("Hi"));
app.listen(3000);
```

Routing

```
app.get/post/put/patch/delete(path, ...handlers)
app.route("/x").get(a).post(b);   app.all("/x", fn)
app.use("/api", router);  // Router({mergeParams:true})
req.params.id · req.query.q · req.body.name
```

Response

```
res.send(x) · res.json(o) · res.status(201).json(o)
res.sendStatus(204) · res.redirect("/x")
res.render("view", data) · res.sendFile(abs)
res.set(h, v) · res.cookie(n, v, opts)
```

Middleware

```
const mw = (req, res, next) => { ...; next(); };
app.use(mw);  app.get("/x", mw, handler);
// error handler (4 args) - LAST
app.use((err, req, res, next) =>
  res.status(err.status || 500).json({ error:
err.message }));
app.use((req, res) => res.status(404).json({ error:
"Not found" }));
```

Popular packages

```
helmet() · cors() · morgan("dev") · compression()
cookieParser() · rateLimit({windowMs, limit})
multer({dest}) · express-session · passport
zod / express-validator · jsonwebtoken · bcrypt
```

Order of middleware

helmet → cors → logger → parsers → static → routes → 404 → error handler

HTTP status codes

200 OK · 201 Created · 204 No content · 301/302 Redirect · 400 Bad request · 401 Unauthenticated · 403 Forbidden · 404 Not found · 409 Conflict · 422 Invalid · 429 Too many · 500 Server error

Golden rules

One response per request · middleware order matters · validate all input · hash passwords · secrets in .env · central error handler · async/await everywhere · thin controllers · separate app from server · test with Supertest · deploy behind HTTPS.

🚀 Build real APIs - that is how you master Express.js! 🚀 — asadokz18